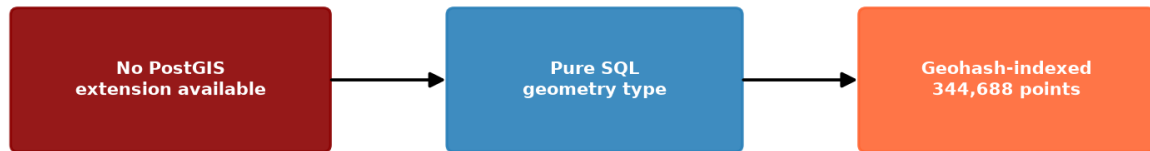


November 2025 - 1

Geospatial Search Without PostGIS



A pure-SQL composite type, geohash indexing, and a real 344,688-row optimization

Figure 1: Figure 0: Geospatial Search Without PostGIS

Welcome to this edition of yugabyteDB Developer’s Notebook (YDN). This month we answer the following question(s);

We have a geospatial application built against PostgreSQL and PostGIS – point-in-radius search, polygon containment, distance calculations, the works. yugabyteDB doesn’t support the PostGIS extension. Does that mean geospatial search is simply off the table if we migrate ?

No – but it does mean rethinking the implementation, not just relying on an extension. This month’s article walks through a real project that reimplements the PostGIS surface area needed for production geospatial search as pure SQL: a composite type standing in for PostGIS’s geometry/geography columns, geohash-based spatial indexing standing in for GiST indexes, and a real, measured optimization against a 344,688-row dataset that took one query from a 5.2-second full scan down to an index-accelerated lookup.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2026.1.1.0. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB’s managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

1.1 Terms and core concepts

1.1.1 Introduction: Replacing an Extension With a Composite Type

PostGIS is a PostgreSQL extension – it is not part of core PostgreSQL, and yugabyteDB, while broadly PostgreSQL-compatible at the SQL layer, does not support it. Rather than treat that as a dead end, this project reimplements the specific PostGIS surface a real application needs as ordinary SQL: a custom composite type, PL/pgSQL functions, and geohash-based indexes – no C extensions, nothing outside what any yugabyteDB cluster already supports.

Example 1-1: The core composite type

```
CREATE TYPE geometry AS (  
  lon double precision[],  
  lat double precision[]  
);
```

A point is stored as single-element arrays; a polygon (or a bounding box) as parallel arrays of vertices, in order. Accessing fields uses ordinary composite-type syntax: `(geom).lon`, `(geom).lat`, `(geom).lon[1]` for the first longitude value. Constructor functions match PostGIS's own naming so the SQL calling code reads familiarly: `ST_MakePoint(lon, lat)`, `ST_MakePolygon(lon[], lat[])`, `ST_MakeEnvelope(lon_min, lat_min, lon_max, lat_max)`.

1.1.2 Two Types, Two Different Kinds of Math

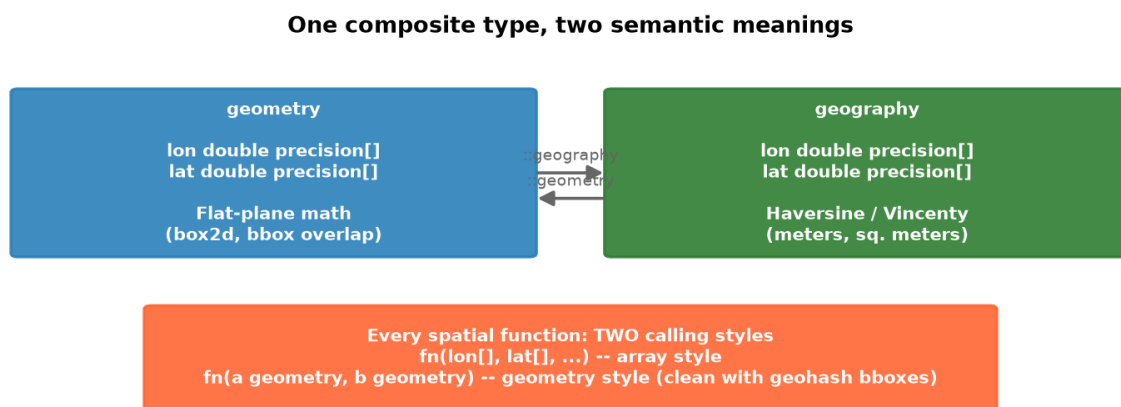


Figure 2: Figure 1: One composite type, two semantic meanings

A second type, **geography**, is structurally identical to **geometry** (the same parallel `lon[]/lat[]` arrays) – the difference is entirely semantic. Functions accepting **geometry** do flat-plane math; functions accepting **geography** treat coordinates as points on a sphere or ellipsoid and return results in meters or square meters. Implicit casts exist both directions (`geom::geography`, `geog::geometry`), so the same underlying data can be reasoned about either way depending on which function is called.

geography distance calculations support two models, a real, deliberate accuracy/speed tradeoff rather than one “correct” answer:

- **Haversine** – treats the earth as a perfect sphere ($R = 6,371$ km). Fast, roughly 0.3% error.
- **Vincenty** – iterative, against the WGS84 ellipsoid. Sub-millimeter accuracy, more expensive per call.

Example 1-2: Choosing between them explicitly

```
-- Haversine (fast, default):  
SELECT ST_Distance(geog_a, geog_b);
```

```
-- Vincenty (accurate, explicit third argument):
SELECT ST_Distance(geog_a, geog_b, true);
```

Every geography-aware function this project provides – `ST_Distance`, `ST_DWithin`, `ST_DistanceSphere`, `ST_DistanceSpheroid`, `ST_Length`, `ST_Perimeter`, `ST_Area`, `ST_Project` – follows this same pattern: a fast default, an explicit opt-in to the more expensive, more precise calculation.

1.1.3 Every Function, Two Calling Styles

A design choice worth calling out directly: every spatial function in this project supports both an “array style” (the original interface, operating directly on `lon[]/lat[]` arrays) and a “geometry style” (operating on the composite type):

Example 1-3: The same containment check, two ways

```
-- Array style:
SELECT ST_Contains(
  ARRAY[-112.0, -111.8, -111.8, -112.0],
  ARRAY[40.4, 40.4, 40.6, 40.6],
  ARRAY[-111.95, -111.85, -111.85, -111.95],
  ARRAY[40.45, 40.45, 40.55, 40.55]);

-- Geometry style:
SELECT ST_Contains(
  ST_MakePolygon(ARRAY[-112.0, -111.8, -111.8, -112.0],
    ARRAY[40.4, 40.4, 40.6, 40.6]),
  ST_MakePolygon(ARRAY[-111.95, -111.85, -111.85, -111.95],
    ARRAY[40.45, 40.45, 40.55, 40.55]));
```

The geometry style is particularly clean when combined with this project’s geohash decoding functions – `geohash_decode_bbox_geom('9x0qs0')` returns a rectangle geometry directly, so a containment check between two geohash cells reads as a single, direct function call rather than four parallel arrays threaded through by hand.

[!NOTE] Neither calling style is deprecated in favor of the other – they coexist because different callers benefit from different shapes: existing code ported from a system that already worked in raw coordinate arrays keeps working unchanged, while new code gets the more readable geometry-style interface.

1.1.4 Geohash Indexing: Standing in for GiST

PostGIS’s spatial performance normally comes from a GiST index over a native geometry column – not available here, since this project’s `geometry` type is an ordinary composite, not something the index machinery understands spatially. Its replacement is geohash: every row carries a `geo_hash10` (full precision) and a `geo_hash8` (coarser, shorter) string column, both ordinary `TEXT`, both indexable with an ordinary B-tree index. Four indexes exist in the real schema: on `geo_hash10`, on `geo_hash8`, and on the first 5 and 6 characters of the geohash prefix – letting a query choose how coarse or fine a spatial bucket to filter on before doing any exact geometric math.

Fourteen geohash utility functions provide the actual spatial reasoning on top of these plain string columns: `geohash_encode(lat, lon, precision)` turns coordinates into a geohash string; `geohash_neighbors(geohash)` returns all 8 surrounding cells as JSONB; `geohash_decode_bbox(geohash)` / `geohash_decode_bbox_geom(geohash)` turn a geohash back into a bounding box, as either raw columns or a geometry; `geohash_in_list_within_miles(geohash, miles)` generates the list of cells covering a given radius, ready for an `IN (...)` clause. Each of these, like the PostGIS-equivalent functions above, has both a table-returning and a geometry-returning form.

The real dataset backing this project’s testing is 344,688 points of interest – loaded, geometry-populated, and geohash-backfilled as a real, sized dataset, not a toy handful of rows, which is what makes the performance

story in the next section a genuine measurement rather than a hypothetical.

1.2 Complete the following

In this section we build and run the real optimization case this project is built around – a genuine query pattern (“find the 10 nearest points within 1km”) that a naive implementation runs as a 344,688-row sequential scan, and a geohash-accelerated rewrite that doesn’t.

1.2.1 Prerequisites

A yugabyteDB cluster, and psql access. The full schema and 344,688-row dataset from this project’s 20 - sql/ folder (10 through 35, run in that numeric order) to reproduce the exact numbers below; a smaller synthetic dataset will demonstrate the same mechanism at a smaller scale.

1.2.2 Step 1 – Stand up the schema, types, and data

Example 1-4: Execution order

```
10_CreateGeometryType.sql      -- geometry composite type + constructors
11_CreateSchema.sql            -- my_mapdata table, 4 geohash indexes
12_CreateGeographyType.sql     -- geography composite type, distance functions
15_LoadData.sql                -- loads 344,688 rows, populates geom + geo_hash8
20_GeohashFunctions.sql        -- 14 geohash utility functions
25_GeometryFunctions.sql       -- PostGIS-equivalent ST_* functions
30_GeohashPolygonFunctions.sql -- geohash8_fully_within_polygon
35_TestQueries.sql             -- worked examples, both calling styles
```

1.2.3 Step 2 – Run the naive version, and see the real cost

Example 1-5: The original query (“Jim’s Query”) – find 10 nearest points within 1000m

```
EXPLAIN (ANALYZE, VERBOSE, DIST, DEBUG)
SELECT md_pk, md_name, md_address, md_city,
       ST_Distance(geom::geography,
                  ST_SetSRID(ST_MakePoint(-105.0775, 40.5853), 4326)::geography, true) AS dist_m
FROM my_mapdata
WHERE ST_DWithin(geom::geography,
                ST_SetSRID(ST_GeomFromText('POINT(-105.0775 40.5853)'), 4326)::geography, 1000, true)
  AND geom::box2d <-> ST_MakeEnvelope(-105.09, 40.57, -105.06, 40.60)::box2d = 0
ORDER BY ST_Distance(geom::geography,
                    ST_SetSRID(ST_MakePoint(-105.0775, 40.5853), 4326)::geography, true)
LIMIT 10;
```

Run this against the full 344,688-row table and inspect the EXPLAIN output: both ST_DWithin and the box2d <-> comparison operate directly on the computed geom expression, which no index can accelerate. The result is a full sequential scan of all 344,688 rows, with a Vincenty distance calculation (the expensive, iterative model from section 1.1.2) run against every single one – confirmed, in this project’s own testing, at roughly 5.2 seconds.

1.2.4 Step 3 – Run the geohash-accelerated version

Example 1-6: The two-phase rewrite

```
EXPLAIN (ANALYZE, VERBOSE, DIST, DEBUG)
WITH nearby_cells AS (
  SELECT unnest(ARRAY[
    geohash_encode(40.5853, -105.0775, 8),
```

Jim's Query: 344,688-row Seq Scan -> geohash two-phase filter

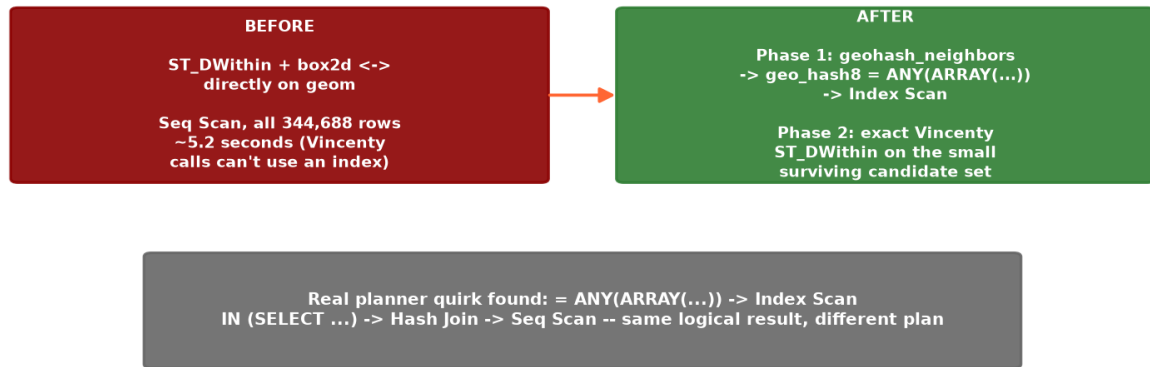


Figure 3: Figure 2: Jim's Query – 344,688-row Seq Scan -> geohash two-phase filter

```
(geohash_neighbors(geohash_encode(40.5853, -105.0775, 8))->>'n'),
(geohash_neighbors(geohash_encode(40.5853, -105.0775, 8))->>'ne'),
(geohash_neighbors(geohash_encode(40.5853, -105.0775, 8))->>'e'),
(geohash_neighbors(geohash_encode(40.5853, -105.0775, 8))->>'se'),
(geohash_neighbors(geohash_encode(40.5853, -105.0775, 8))->>'s'),
(geohash_neighbors(geohash_encode(40.5853, -105.0775, 8))->>'sw'),
(geohash_neighbors(geohash_encode(40.5853, -105.0775, 8))->>'w'),
(geohash_neighbors(geohash_encode(40.5853, -105.0775, 8))->>'nw')
]) AS cell_hash
)
SELECT md_pk, md_name, md_address, md_city,
       ST_Distance(geom::geography,
                  ST_SetSRID(ST_MakePoint(-105.0775, 40.5853), 4326)::geography, true) AS dist_m
FROM my_mapdata
WHERE
  -- Phase 1: geohash index scan (ix_mapdata_geo_hash8)
  geo_hash8 = ANY(ARRAY(SELECT cell_hash FROM nearby_cells))
  -- Phase 2: exact Vincenty refinement, only on Phase 1's survivors
  AND ST_DWithin(geom::geography,
                 ST_SetSRID(ST_MakePoint(-105.0775, 40.5853), 4326)::geography, 1000, true)
ORDER BY dist_m
LIMIT 10;
```

Phase 1 computes the center geohash-8 cell for the search point plus its 8 neighbors (a 3x3 grid of cells, comfortably covering a 1000m radius at this precision), then filters `my_mapdata` down to only rows in one of those 9 cells – a query `ix_mapdata_geo_hash8` can actually serve as an Index Scan. Phase 2 then runs the same exact, expensive Vincenty `ST_DWithin` check as before, but only against the small candidate set that survived Phase 1, rather than against all 344,688 rows.

1.2.5 Step 4 – Confirm the specific planner quirk

Example 1-7: The construction that matters

```
-- Produces an Index Scan on ix_mapdata_geo_hash8:
WHERE geo_hash8 = ANY(ARRAY(SELECT cell_hash FROM nearby_cells))

-- Produces a Hash Join -> Seq Scan instead, same logical result:
WHERE geo_hash8 IN (SELECT cell_hash FROM nearby_cells)
```

Run both forms with `EXPLAIN` and compare plans directly – confirmed in this project’s own testing, the planner treats these two logically equivalent constructs differently. `= ANY(ARRAY(...))` materializes the subquery result as a literal array the index can be probed against directly; `IN (SELECT ...)` is planned as a join, which for this small, cheap subquery (9 geohash strings) still ends up preferring a sequential scan of the large table as the join’s other side. Getting the two-phase design’s real benefit depends on using the specific SQL form the planner can actually turn into an Index Scan, not just any logically equivalent rewrite.

Section summary In this section, we reproduced a real geospatial performance problem at real scale – a 5.2-second, 344,688-row sequential scan caused by an expensive geography calculation that no index could accelerate directly – and fixed it with a two-phase geohash filter: a cheap, index-accelerated coarse pass followed by the same exact expensive calculation applied only to a small surviving candidate set. We also confirmed a specific, non-obvious planner behavior (`= ANY(ARRAY(...))` versus `IN (SELECT ...)`) that determines whether the accelerated version actually gets the Index Scan it’s designed around.

1.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- A pure-SQL replacement for the PostGIS extension, built as ordinary composite types (`geometry`, `geography`) and PL/pgSQL functions – no C extensions required, and no dependency on a spatial index type yugabyteDB doesn’t support.
- The `geometry/geography` split and its real reason for existing: identical underlying storage, different math (flat-plane versus spherical/ellipsoidal), with an explicit fast-versus-accurate choice (Haversine versus Vincenty) exposed directly in each geography function’s signature.
- Geohash-based indexing as GiST’s replacement – 14 utility functions and 4 plain B-tree indexes over ordinary text columns, providing spatial bucketing without any spatially-aware index type.
- A real, measured optimization against a 344,688-row dataset: a naive geography query that ran as a 5.2-second full scan, rewritten as a two-phase geohash filter, plus the specific SQL construct (`= ANY(ARRAY(...))` versus `IN (SELECT ...)`) that determines whether the rewrite actually gets the Index Scan it depends on.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely’s very excellent blog site,

<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2025-11%20-%20Geospatial%20Search%20Without%20PostGIS.pdf>